

FINAL YEAR ENGINEERING PROJECT · TECHNICAL EXPLAINER

DeepShieldAI

Explainable Multimodal AI Framework for Deepfake
Detection and Digital Content Authenticity
Verification

A single reference document covering what the system does, how every layer of the stack works, why each technology was chosen over the alternatives, and how to defend every design decision and every number in front of an examiner.

Prepared for technical round / viva preparation · September 2026

1. What the project is, in one paragraph

DeepShieldAI is a full-stack web application that takes a piece of digital content — an image, an audio clip, a video clip, a news/text article, a product review, or a social-media account's profile metadata — and answers two questions: **is it fake** (deepfake, AI-generated, synthetic, bot-operated), and **why does the model think so**. The second question is the whole point of the "Explainable" in the title: a black-box "87% fake" verdict is not useful to a moderator or an investigator who has to justify a decision, so every prediction ships with a modality-appropriate explanation — a heatmap, a token highlight, a feature-attribution chart — generated by a separate explainability layer sitting on top of the detection models.

The system is built as six independently trained ML detectors behind one common API contract, a FastAPI backend that owns authentication/storage/orchestration, a React frontend for upload/verdict/explanation/history, and (from Phase 11) an asynchronous task queue so a slow analysis never blocks the rest of the application. Every architectural choice below exists because of a concrete problem it solves — that mapping from *problem* → *decision* is what an interviewer is really testing when they ask "why did you use X".

1.1 Why this problem matters

Deepfakes and synthetic content are no longer a novelty concern — they are used for fraud, disinformation, fake reviews that mislead purchasing decisions, and bot-driven manipulation of social platforms. A *single*-modality detector (image-only, say) misses the fact that misinformation campaigns are multimodal: a fabricated video comes with a fabricated article and a bot network amplifying it. DeepShieldAI's premise is that a authenticity-verification tool should cover as many content types as the threat actually spans, under one consistent verdict/explanation contract, rather than being five unrelated single-purpose scripts.

2. System architecture

2.1 High-level component diagram



2.2 The layering rule inside the backend

Every backend feature follows the same four-layer path, strictly in one direction — `routes → services → repositories → models`:

Layer	Responsibility	Never does
routes (<code>app/api/routes/*.py</code>)	Parse the HTTP request, check auth/role, call exactly one service method, shape the response with a Pydantic schema.	Never talks to the database or a model directly.

services (<code>app/services/*.py</code>)	Business logic: "what does 'analyze a document' actually involve" — resolve the detector, create rows, call the repository, handle domain errors.	Never builds SQL or knows about HTTP status codes.
repositories (<code>app/repositories/*.py</code>)	The only layer that writes SQLAlchemy queries — one repository per model.	Never contains business rules ("is this document type supported").
models (<code>app/models/*.py</code>)	SQLAlchemy ORM table definitions — the shape of the data at rest.	Never contains logic beyond simple computed properties.

This is a textbook layered/clean architecture, and the reason to defend it in an interview is testability and blast-radius: a bug in a SQL query is isolated to one repository file; swapping Postgres for another database only touches the repository layer; a route can be unit-tested by mocking one service call instead of standing up a database.

3. Technology stack, and why each piece was chosen

3.1 Backend

Tool	Role	Why this, not the obvious alternative
FastAPI	HTTP API framework	Async-native (important once slow ML inference and Celery enqueue calls are in the request path), automatic OpenAPI/Swagger docs generated from the same Pydantic models used for validation (no hand-written API spec to drift out of sync), and Pydantic type-checking on every request/response — versus Flask, which needs three extra libraries (marshmallow/flask-smorest for validation, an async extension) to reach the same starting point.
PostgreSQL + SQLAlchemy + Alembic	Relational storage + ORM + migrations	The domain is inherently relational (a Document has many Predictions, a Prediction has an Explanation, a User has a Role) — a document database would just be reinventing joins in application code. SQLAlchemy's ORM keeps the repository layer readable; Alembic gives a versioned, reviewable history of schema changes instead of manual <code>ALTER TABLE</code> scripts run by hand.
JWT auth + role-based access control	Authentication/authorization	Stateless tokens mean the API doesn't need a server-side session store to scale horizontally later; role checks (e.g. admin-only routes for audit logs) are enforced as a FastAPI dependency, so the check lives in one place instead of being copy-pasted into every route handler.
Celery + Redis (Phase 11)	Asynchronous task queue for inference	See §7 — this exists specifically because Audio/Video inference on CPU takes multiple seconds and a synchronous request handler would block every other user's request for that whole duration on Uvicorn's default single-worker event loop.

3.2 Frontend

Tool	Why
React 18	Component reuse across five near-identical "upload → verdict → explanation" flows (one per modality) — a single <code>Predict</code> page and a single <code>Explain</code> page branch on <code>document_type</code> rather than five duplicated pages.
Vite	Dev-server startup and hot-reload measured in milliseconds versus webpack's bundling-on-every-change; its dev proxy also solves CORS during local development by forwarding <code>/api</code> to the FastAPI backend.
Tailwind-style utility CSS	Styling co-located with markup for a dashboard with many small, similarly-styled cards/badges/panels — avoids a separate, ever-growing CSS file and naming-convention debates (BEM etc.) for a solo-developer project.

3.3 Machine learning stack (per modality)

Modality	Model	Framework	Why this architecture
Image	MobileNetV2 (transfer learning, frozen backbone)	Keras 3 (on a PyTorch backend, not TensorFlow)	MobileNetV2 is a small, ImageNet-pretrained CNN well suited to a limited-data, limited-compute fine-tuning setup (1,000 labelled images) — training a CNN from scratch would need orders of magnitude more data. Keras 3 was picked for a fast, high-level training loop; running it on the PyTorch backend was a deliberate compatibility decision, not a preference — TensorFlow has no published wheel for Python 3.14 (the project's runtime), so <code>KERAS_BACKEND=torch</code> lets the exact same Keras code execute on the PyTorch install every other detector already needs, with zero TensorFlow dependency.
Audio	wav2vec2-base (fine-tuned)	HuggingFace Transformers / PyTorch	wav2vec2 is pretrained via self-supervised learning on raw audio waveforms, so it already encodes rich acoustic representations before fine-tuning — a strong prior for a 1,000-clip dataset that would be far too small to train a spectrogram CNN from scratch. This is the project's best-performing model (F1 0.98).
Video	R3D-18 (3D ResNet, Kinetics-)	torchvision / PyTorch	A 3D CNN convolves across space <i>and</i> time, which a frame-by-frame 2D CNN cannot — deepfake artifacts often show up as temporal flicker between frames, not within any single frame. R3D-18 was chosen as the smallest widely-available pretrained 3D architecture. <i>(Disclosed honestly in §6: with only</i>

	pretrained)		106 training clips, this 34M-parameter model did not learn — see the limitations table.)
Text / News	DistilBERT (fine-tuned)	HuggingFace Transformers / PyTorch	DistilBERT keeps ~97% of BERT's language understanding at roughly half the parameters and inference latency — the right trade-off for a real-time API endpoint rather than a batch offline job.
Review	DistilBERT (fine-tuned, separate checkpoint)	HuggingFace Transformers / PyTorch	Same architecture as Text, but trained on a different task and dataset (computer-generated vs. human-written reviews) — kept as a separate model rather than a shared multi-task head, because the two tasks have different label semantics and training data sizes.
Bot / Account	XGBoost (gradient-boosted trees)	XGBoost + scikit-learn (training only)	Account metadata (follower counts, ratios, account age, profile-field flags) is structured tabular data with non-linear feature interactions — the textbook case where gradient-boosted trees outperform a neural network, need far less data (2,432 rows was enough for F1 0.926), and — critically for this project — are natively explainable via SHAP's <code>TreeExplainer</code> , giving exact, not approximate, feature attributions.

3.4 Explainability (XAI) libraries

Method	Used for	Why
Grad-CAM (hand-implemented, plain <code>torch.autograd</code>)	Image	Grad-CAM back-propagates the gradient of the predicted class into the last convolutional layer to produce a coarse heatmap of "where the model looked." It needed no extra library — implemented directly with PyTorch autograd hooks — because the standard TensorFlow implementation (<code>tf-keras-vis</code>) doesn't apply to a Keras-on-PyTorch model.
SHAP	Text, Review, Bot/Account	SHAP gives theoretically grounded, additive feature attributions (Shapley values from game theory) — for the tree-based Bot detector specifically, <code>TreeExplainer</code> computes <i>exact</i> attributions in polynomial time, not a sampling approximation.
LIME	Image (secondary), Text/Review (secondary)	LIME explains one prediction at a time by perturbing the input (occluding image super-pixels, dropping words) and fitting a local linear model — a useful cross-check against Grad-CAM/SHAP because it makes no assumption about model internals (model-agnostic), unlike Grad-CAM which needs gradient access.
Waveform saliency (custom)	Audio	There is no pixel grid to draw a heatmap over, so the explanation is a time-window importance curve overlaid on the waveform — honest about being a 1-D signal, not a "fabricated" spectrogram image dressed up to look like Grad-CAM.
3-D Grad-CAM frame attribution (custom)	Video	Extends Grad-CAM through the temporal dimension of R3D-18 and reports which <i>sampled frames</i> contributed most to the verdict, rather than one flattened heatmap that would misleadingly imply a single spatial region across the whole clip.

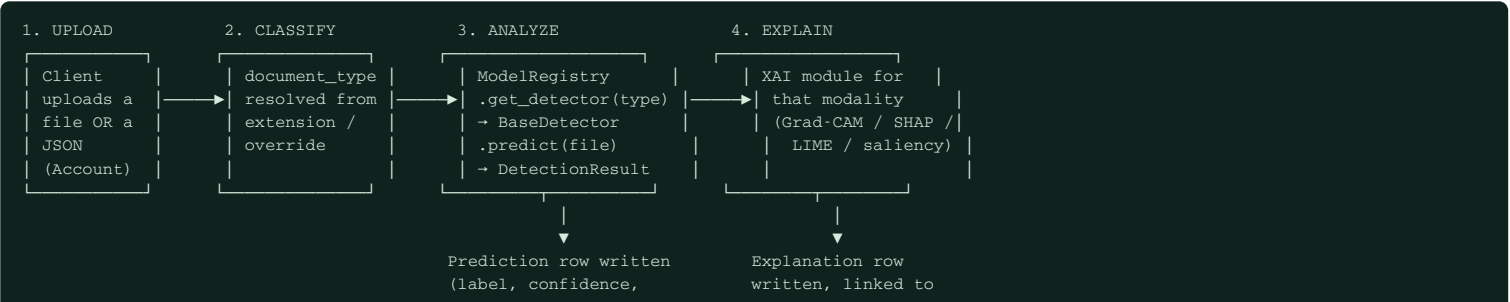
Why explainability is a first-class module, not an afterthought: a raw softmax score is not actionable for a human reviewer — "94% fake" doesn't tell a moderator whether to trust it. Grad-CAM/SHAP/LIME turn a black-box score into a falsifiable claim ("the model focused on this facial region / this feature / this phrase"), which is what lets a human sanity-check the model and catch failure modes like the Review model's OOD weakness documented in §6.

3.5 Retrieval layer (RAG)

A separate knowledge-base feature lets a user upload trusted reference documents (guidance on how to spot deepfakes, dataset documentation, etc.), which are chunked, embedded with `all-MiniLM-L6-v2` (a compact sentence-transformer — 384-dim embeddings, fast enough for CPU-only indexing), and stored in **ChromaDB** (an embedded vector database — no separate server process to run, unlike Pinecone/Weaviate, which matters for a project meant to run on one laptop). A query does a semantic similarity search and returns the nearest passages with source/page metadata. **Important distinction to state precisely in an interview:** this is retrieval only — it returns ranked source passages, not an LLM-generated synthesized answer. Calling it "RAG" is accurate (Retrieval-Augmented — the retrieval half), but there is deliberately no generation step wired to an LLM yet.

4. The core data flow: Document → Prediction → Explanation

Every one of the six modalities — despite having six completely different models — is driven through the *same three-table pipeline*, which is the single most important architectural decision in the project to be able to explain clearly:



```
model_name, timing,    the Prediction it
status)                explains
```

4.1 BaseDetector — the contract every model implements

All six detectors — despite using three different ML frameworks (Keras, PyTorch/Transformers, XGBoost) — implement the same abstract interface:

```
class BaseDetector(ABC):
    modality: str          # "Image", "Audio", "Video", "Text", "Review", "Account"
    model_name: str
    is_ready: bool         # False if weight files are missing — API still boots

    def load(self) -> None: ...
    def predict(self, file_path: str) -> DetectionResult: ...
    def describe(self) -> dict: ...

@dataclass(frozen=True)
class DetectionResult:
    label: str
    confidence: float
    probabilities: dict
    model_name: str
    processing_time: float
    metadata: dict
```

This is the Strategy design pattern: the service layer that runs a prediction never branches on "if Image do X, if Audio do Y" — it just calls `detector.predict(file_path)` on whatever detector the registry resolved. Adding a seventh modality later means writing one new class that satisfies this interface; nothing else in the request path changes.

4.2 ModelRegistry — one load, process-wide reuse

Loading Audio's wav2vec2 checkpoint from disk takes real time and ~380MB of memory. Doing that on every request would be unacceptably slow, so `ModelRegistry` is a process-wide singleton that loads all six detectors exactly once, at process startup (FastAPI's `lifespan` hook), guarded by double-checked locking so concurrent requests during startup can't trigger a duplicate load. If a model's weight files are missing, that one detector reports `is_ready=False` instead of crashing the whole API — every other modality keeps working.

4.3 The "which document_type is this?" problem — a real bug this project fixed

Two modalities — Text and Bot/Account — can both arrive as a plain `.json` file, so the file extension alone can't disambiguate them. The fix was an explicit `document_type` override form field the frontend sends on upload (the Bot form builds its JSON client-side and knows what it is), plus a `document_type_name` computed property added to the `Document` ORM model (a plain Python `@property`, not a database column) so the API response schema can expose the resolved type name without a schema migration. This is a good example of a small, deliberate design choice that avoids a much bigger one (a schema change) for a problem that a schema change wasn't actually needed to solve.

5. Explainability, in more depth

5.1 Grad-CAM walkthrough (Image)

1. Run a forward pass through MobileNetV2 up to its last convolutional feature map (before global average pooling).
2. Backpropagate the gradient of the predicted class's output score with respect to that feature map.
3. Global-average-pool those gradients per channel — this gives each channel an "importance weight" for this specific prediction.
4. Compute a weighted sum of the feature-map channels, apply ReLU (only positive influence matters), and upsample to the input image's resolution.
5. Overlay the resulting heatmap on the original image.

Common interview trap: Grad-CAM explains "where the network looked", not "why it is right" — a model that looks at the correct face region can still be wrong, which is exactly the Image detector's documented ~75% accuracy ceiling (§6).

5.2 SHAP for tabular data (Bot/Account) — exact, not approximate

For tree ensembles specifically, SHAP's `TreeExplainer` computes exact Shapley values by exploiting the tree structure (polynomial time via the tree's conditional expectations), rather than the model-agnostic `KernelExplainer`'s sampling approximation used for the transformer-based Text/Review models. Being able to state this distinction — exact for trees, approximate/sampled for black-box models — is a strong signal of real understanding, not just "I used SHAP".

5.3 LIME in one sentence

LIME perturbs the input locally (mask super-pixels in an image, delete words in a sentence), observes how the model's output changes, and fits an interpretable linear model to those local perturbations — the coefficients of that local linear model are the explanation. It is model-agnostic (works on any black box) but its explanations are only locally faithful, not globally.

6. The six detectors: metrics and honest limitations

Every number below is read directly from the training notebooks' captured output — not estimated. Being able to recite these exactly, including the weak ones, is one of the strongest things you can do in a viva: it signals the results weren't just copy-pasted from a tutorial.

Modality	Test F1	Test set (n)	Status	Headline caveat
Audio (wav2vec2)	0.980	150	Live	Strongest model — clean, complete pipeline (checkpoint includes its own preprocessor config).
Bot/Account (XGBoost)	0.926	365	Live	Trained only on Twitter/X-shaped accounts; "sentiment" feature is bio sentiment, not tweet sentiment (documented, not hidden).
Review (DistilBERT)	0.973	6,063	Live	Large, trustworthy test set — but fails on obvious hand-written spam the training distribution never saw (learned <i>this dataset's</i> synthetic style, not "fakeness" generally).
Text/News (DistilBERT)	0.982*	335	Wired, flagged	*40% of the raw dataset silently vanished during label encoding, collapsing the class balance from 50/50 to 83/17 — the headline number is inflated by that imbalance.
Image (MobileNetV2)	0.782 (n=100 notebook set)	100 / 80	Live	Independently re-verified at ~75% accuracy on a separate 80-image sample; roughly 1 in 4 verdicts is simply wrong, and the sigmoid is uncalibrated near the decision boundary.
Video (R3D-18)	0.385	11	Broken	Predicts "real" for every single input — 84 training clips was not enough data for a 33M-parameter, fully-unfrozen 3D CNN. Validation accuracy diverged across epochs.

Why disclosing the Video and Text problems openly is the right engineering call, and the right interview answer: a model with F1 0.385 that predicts one class for every input has not learned anything — deploying it as "working" would be actively misleading. The correct fix (freeze the backbone, gather far more data, or drop the modality and route video frames through the Image detector instead) is *additional scope*, not a five-minute patch, and this project's documentation says so explicitly rather than quietly rounding the number up. If an examiner asks "does your video detector work" — the honest, confident answer is "no, and here is exactly why, and here is what would fix it" — that is a stronger answer than a vague claim of success that falls apart under one follow-up question.

6.1 The Image-detector debugging story (good "tell me about a bug you fixed" material)

A user reported two genuine photos both being confidently classified as "Deepfake." Rather than guessing at a fix, the investigation empirically tested all 3 preprocessing scalings (`mobilenet_v2` [-1,1], `rescale` [0,1], `raw`) crossed with both sigmoid polarities — 6 combinations — against 40 real + 40 fake labelled images. Results:

Preprocessing mode	Mean score (real)	Mean score (fake)	Best accuracy
<code>mobilenet_v2</code>	0.71	0.84	53.8%
<code>rescale</code> (deployed)	0.40	0.73	75.0%
<code>raw</code>	0.74	0.73	50.0% (coin flip)

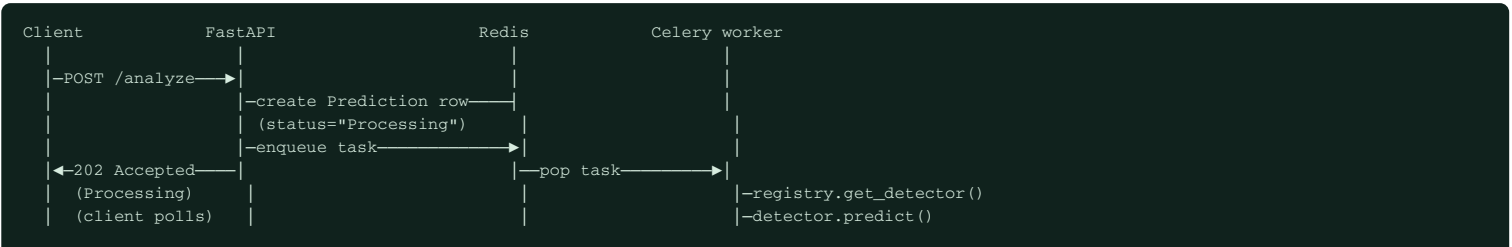
The deployed configuration won outright, and `raw` landing at exactly a coin flip is useful negative evidence that `rescale` is genuinely doing something rather than being the least-bad guess. **Conclusion: the configuration was never the bug — the underlying ~75% model accuracy is a genuine, disclosed ceiling**, and the two real fixes (fine-tuning on more/better data, or temperature-scaling the confidence) were deliberately left as future work rather than being rushed in in a way that could introduce a regression this late in the project.

7. Asynchronous inference pipeline (Celery + Redis)

7.1 The problem it solves

Image and the XGBoost Bot detector return in well under a second. Audio (wav2vec2) and Video (16-frame R3D-18) routinely take several seconds on CPU-only hardware. Uvicorn's default worker handles one request at a time — without a queue, one user's slow Video analysis blocks every other request to the API for its full duration, including an unrelated user just trying to load their history page.

7.2 How it works



```
| -GET /predictions/id> | | | -update row (Completed/Failed)
| ←still Processing-| | |
| ... repeats every 1.5s ... | | |
| -GET /predictions/id> | | |
| ←Completed + result| | |
```

7.3 Design decisions worth defending individually

Decision	Reasoning
Redis as broker only , no result backend	The <code>predictions</code> table in Postgres is already the durable record of a task's outcome. A Celery result backend (often Redis too) would just be a second place that same fact could live — and the two could disagree. One source of truth, always.
Models load once per worker process , via <code>worker_process_init</code>	A Celery worker is a separate OS process from Uvicorn — it does not share the API process's <code>ModelRegistry</code> singleton (singletons are per-process, not per-application). Loading inside the task function instead would reload Audio's 380MB checkpoint on <i>every single prediction</i> — slower than the synchronous code this phase replaces.
Detector resolved before the Prediction row is created	<code>start_analysis()</code> calls <code>registry.get_detector()</code> synchronously in the request handler, before writing anything. An unsupported/unloaded modality fails the HTTP request immediately with a 503 — it never becomes a queued task that silently fails minutes later with nobody watching.
<code>max_retries=0</code> — no automatic retries	A model failing on a specific file (corrupt video, missing weights) does not self-heal on a second attempt. Retrying would only delay the user seeing "Failed" by the retry backoff, for zero benefit.
Frontend polls the same existing REST endpoint	No websocket, no server-sent events, no new endpoint — <code>GET /predictions/{id}</code> is the same read-only route the History page already used; polling it every 1.5s until the status changes is the smallest change that gets the job done.

`--pool=solo` is required to run the Celery worker on Windows, because Celery's default worker pool relies on `os.fork()`, which Windows does not implement.

8. Security and the database domain model

8.1 Authentication and authorization

- **JWT bearer tokens** issued at login, verified on every protected route via a FastAPI dependency — no server-side session store, so the API can scale horizontally without sticky sessions.
- **Role-based access control** — role checks are a reusable dependency, not duplicated `if` checks in each route (e.g. audit-log routes are admin-only).
- **Audit log** — a dedicated table/route records sensitive actions, so "who did what, when" is answerable after the fact — an expected feature in any system that makes authenticity/moderation decisions.

8.2 Core tables (domain model)

Table	Purpose
<code>users / roles</code>	Accounts and their permission level.
<code>document_types</code>	Reference data — the six supported modalities and their display names (this is what <code>document_type_name</code> reads from, §4.3).
<code>documents</code>	One row per uploaded file/JSON snapshot: storage path, type, uploader, timestamp.
<code>predictions</code>	One row per analysis run: label, confidence, model name, processing time, status (Processing/Completed/Failed) — the single source of truth for a task's outcome (§7.3).
<code>explanations</code>	Linked to a Prediction; stores the XAI output (heatmap path, SHAP values, token attributions) for that specific verdict.
<code>audit_logs</code>	Sensitive-action trail, admin-readable only.

Migrations are managed with Alembic, run from inside `backend/` — every schema change is a reviewable, ordered migration file rather than a manual `ALTER TABLE`, which matters the moment more than one person (or more than one environment — dev/staging/prod) needs the same schema.

9. Known limitations — full list (say these before you're asked)

- **Video** The R3D-18 detector does not work — it predicts one class for every input due to a training set of only 106 clips for a 33M-parameter model with the entire backbone unfrozen.
- **Text/News** The training notebook's label-encoding step silently dropped 40% of rows and collapsed the class balance from 50/50 to 83/17 — the reported 99.4% accuracy needs that caveat attached every time it's quoted.
- **Image** ~75% real-world accuracy with an uncalibrated decision boundary — independently re-verified, not a configuration bug — roughly 1 in 4 verdicts is wrong.
- **Review** 97.3% F1 on held-out data, but fails on obvious hand-written spam outside the training distribution — it learned this dataset's

synthetic style, not "fakeness" in general.

- **Bot/Account** Trained only on Twitter/X-shaped account features; the "sentiment" feature is bio sentiment (VADER on the profile description), not tweet sentiment, because the source dataset's tweet dump wasn't reliably parseable.
- **Scope** The RAG feature performs retrieval only — it returns ranked source passages, with no LLM-based answer generation wired up yet.
- **Scope** Explanation generation is still synchronous (not yet moved onto the Celery pipeline) — acceptable today because it runs on-demand, per already-completed prediction, and finishes quickly for most modalities.
- **Scope** No automatic task retries, dead-letter queue, or priority queues in the async pipeline — reasonable production concerns, deliberately out of scope for a single-developer academic project.
- **Scope** No Docker packaging or full automated test suite yet — the next planned phase.

Framing this well in a viva: list these unprompted, in one breath, before the examiner finds them. It reframes "your project has bugs" into "the candidate understands their own system's failure modes and made deliberate, justified scoping decisions" — which is a materially different (and much stronger) impression.

10. Likely interview questions and model answers

Q1. Why did you build six separate models instead of one universal deepfake detector?

Because the input spaces are not compatible — a 2D CNN for pixels, a speech transformer for waveforms, a 3D CNN for spatio-temporal video, a text transformer for tokens, and gradient-boosted trees for structured account metadata are each the right tool for their input's structure. A single "universal" model would need a shared input representation across images, audio, video, text, and tabular data, which is an open research problem, not something the scope of this project could responsibly attempt. The unifying layer instead sits one level up — the common `BaseDetector` contract and `Document → Prediction → Explanation` pipeline (§4) — which is where "one consistent system" actually lives.

Q2. Why is explainability necessary — isn't the accuracy the important part?

Accuracy alone can't be trusted blindly, and this project's own Image and Review models prove why: Review is 97% accurate on held-out data yet fails on obvious spam, and Image is right 75% of the time with an uncalibrated confidence score. A human reviewer looking at a Grad-CAM heatmap or a SHAP feature breakdown can catch a wrong-but-confident verdict that a bare percentage would hide — explainability is a safety mechanism for exactly the failure modes documented in §6, not a cosmetic add-on.

Q3. Why Celery + Redis instead of just using FastAPI's `BackgroundTasks` ?

`BackgroundTasks` still runs inside the same Uvicorn worker process — a long-running Video inference would still tie up CPU/GIL time on the same process handling other requests, and there'd be no separate, restartable worker or task queue if the API process needs to redeploy. Celery gives a genuinely separate process (or, later, separate machine) pulling work off a durable queue, so the API and the inference workload can scale and restart independently.

Q4. Why does the Image detector run on Keras but everything else on PyTorch — doesn't that add complexity?

It's a real trade-off, not a free choice: the trained artefact is a `.keras` file, and Keras 3 is backend-agnostic, so rather than requiring a TensorFlow install (which has no Python 3.14 wheel at all right now) the project sets `KERAS_BACKEND=torch` so the same Keras model executes on the PyTorch runtime every other detector already needs. The cost is one extra mental model at inference time; the alternative (blocking the whole Image modality on an unscheduled TensorFlow release) was worse.

Q5. What was the hardest bug you fixed, and how did you find it?

The Image "real photos classified as Deepfake" report (§6.1) — the fix wasn't a code change at all, it was a disciplined empirical test: cross every preprocessing scaling with every sigmoid polarity against a labelled sample, rather than guessing. The result proved the deployed configuration was already optimal and that the true limit was the model's own ~75% accuracy — the value of the exercise was converting "something feels wrong" into a falsifiable, quantified answer.

Q6. If you had one more month, what would you fix first, and why that one?

The Video detector, because it is the only modality that is completely non-functional (predicts one class always) rather than imperfect. The fix is well understood — freeze the R3D-18 backbone and train only the final classification head, so a 33M-parameter model isn't trying to learn from 84 clips — it's scoped, low-risk work, unlike the Image detector's accuracy ceiling, which needs a materially larger dataset or confidence calibration and is a bigger undertaking.

Q7. How do you know your reported metrics aren't inflated?

By checking, not assuming — which is exactly what caught the Text detector's problem: tracing row counts through the training notebook showed 40% of the dataset vanished during label encoding and the class balance collapsed to 83/17, which inflates a naive accuracy reading. That's now documented as a caveat rather than left for someone else to discover, and it's the reason the project reports F1 alongside accuracy everywhere — F1 is far less forgiving of class imbalance.

Q8. What's a common mistake this project deliberately avoided?

Treating a `.keras / .pth` weights file as if it fully specifies inference. It doesn't — it has no memory of which pixel-scaling convention or which sigmoid polarity training used. Both have to be reverse-engineered from the training notebook (or empirically calibrated) and hardcoded into the serving config; getting the polarity wrong specifically would silently invert every single verdict with no error raised anywhere — the kind of bug that only becomes visible when a real user reports patently wrong results.

11. Development approach and roadmap

The project was built in explicit, sequentially-approved phases rather than as one monolithic build — each phase stated its objective, prerequisites, architecture and data flow before any code was written, was implemented, explained line-by-line, and given testing steps before moving on. This phase-gated approach is itself worth mentioning in an interview as a demonstration of engineering discipline on a solo project, where there is no second reviewer to catch scope creep or an unreviewed architecture decision.

Area	Status
Core CRUD, auth/RBAC, document upload	Complete
Five media detectors (Image/Audio/Video/Text/Review) wired to the generic pipeline	Complete
Explainability (Grad-CAM, SHAP, LIME, waveform/frame attribution)	Complete
Bot/Account detector (XGBoost) + frontend "Analyze an account" flow	Complete
RAG knowledge-base retrieval (ChromaDB + MiniLM)	Complete (retrieval only, no generation)
Asynchronous inference pipeline (Celery + Redis)	Implemented and verified; final rollout in progress
Docker packaging	Planned next phase
Full automated test suite	Planned next phase
LIME for Audio/Video, async explanation generation	Identified follow-on work, not started

Prepared as a technical-round reference for DeepShieldAI — figures and file names verified against the project's own training notebooks and source code as of September 2026. Quote the caveats alongside every metric.